TREZ

Lenguaje de Dominio Específico Funcional

para Deep Learning – Especificación de Diseño

Julián David Cristancho Bustos

y Equipo de Desarrollo

Universidad Sergio Arboleda

16 de marzo de 2026

Índice

1. Introducción **3**

2. Arquitectura del Sistema **3**
2.1. Front-end – Análisis Léxico y Sintáctico (ANTLR4) 3
2.2. Back-end – Motor de Evaluación (Python) 4

3. Alcance del Proyecto (MVP) **4**
3.1. Dentro del Alcance 4
3.2. Fuera del Alcance 5
3.3. Hitos de Desarrollo 6

4. Gramática Tipo 2 (ANTLR4) **6**
4.1. Descripción de las Reglas Productivas 6
4.2. Gramática Actual — Entrega 1 (implementada) 7
4.3. Gramática Objetivo — Entrega 2 (planificada) 8
4.4. Operador Pipe (`|>`) 10

5. Descripción Sintáctica del Lenguaje **10**
5.1. Asignación de Variables 11
5.2. Funciones Lambda 11
5.3. Llamadas a Módulos (*Namespace*) 12
5.4. Literales de Tensor 12
5.5. Pipelines con el Operador `|>` 13

6. Librería Estándar Nativa para Deep Learning **13**
6.1. Módulo `Tensordoz` – Álgebra Lineal 14
6.2. Módulo `Mathdoz` – Matemáticas y Activaciones 14
6.3. Módulo `Optimdoz` – Optimizadores 15
6.4. Módulo `Metricsdoz` – Evaluación 15
6.5. Módulo `IOdoz` – Entrada/Salida 15
6.6. Módulo `Plotdoz` – Visualización 16
6.7. Módulo `Inspectdoz` – Depuración No Invasiva 16

7. Sistema de Manejo de Errores **17**
7.1. Excepciones Definidas 17

8. Ejemplos de Programas Completos en TREZ **18**
8.1. Caso 0: Backpropagation Manual (Entrega 1 – ejecutable hoy) 18
8.2. Caso 1: Entrenamiento de una Red Neuronal MLP 19

8.3. Caso 2: Autoencoder para Reducción de Dimensionalidad 20
8.4. Caso 3: Análisis Exploratorio con Depuración de Pipeline 21

9. Diagramas del AST y el Patrón Visitor **22**
9.1. Etapa 1 – Del Código Fuente al Parse Tree 22
9.2. Etapa 2 – Del Parse Tree al AST 23
9.3. Etapa 3 – Recorrido del AST con el Patrón Visitor 24
9.4. Etapa 4 – Flujo Completo de un Pipeline de Entrenamiento 24

10. Semántica Operacional del Intérprete **25**
10.1. Reglas de Evaluación 25

11. Conclusiones y Trabajo Futuro **26**

1. Introducción

TREZ es un Lenguaje de Dominio Específico (DSL) funcional construido completamente desde cero, diseñado para la definición, entrenamiento y evaluación de modelos de *Deep Learning*. Su característica más distintiva es la **ausencia total de dependencias externas** para el cómputo numérico: no utiliza NumPy, PyTorch, TensordozFlow ni ninguna librería de álgebra lineal.

El lenguaje adopta un **paradigma estrictamente funcional**, donde las transformaciones de datos se expresan mediante el operador de composición `|>` (*pipe*), lo que permite construir pipelines declarativos de entrenamiento que son legibles, reproducibles y libres de efectos secundarios.

El sistema de compilación se apoya en **ANTLR4** para el análisis léxico y sintáctico, y en un intérprete **Python** que implementa el patrón *Visitor* sobre el Árbol Sintáctico Abstracto (AST) generado.

> **Principio de diseño central:** En TREZ, cada expresión es una función. No existen variables mutables ni bucles imperativos. El entrenamiento de una red neuronal es una cadena de transformaciones funcionales puras encadenadas con `|>`.

2. Arquitectura del Sistema

El compilador e intérprete de TREZ operan sobre una arquitectura de dos niveles claramente separados:

2.1. Front-end – Análisis Léxico y Sintáctico (ANTLR4)

El *front-end* es responsable de transformar el texto fuente en una representación estructurada que el intérprete pueda recorrer. Comprende dos componentes:

- **Lexer:** Segmenta el flujo de caracteres en *tokens* atómicos: identificadores (`ID`), literales numéricos (`NUMBER`), cadenas (`STRING`), el operador *pipe* (`|>`), paréntesis, corchetes y separadores.
- **Parser:** Aplica la gramática Tipo 2 (Libre de Contexto) definida en el archivo `TREZ.g4` para construir el *Parse Tree*, del cual ANTLR4 genera automáticamente las clases *Listener* y *Visitor* en Python.

2.2. Back-end – Motor de Evaluación (Python)

El *back-end* recorre el AST mediante el patrón *Visitor*. Cada nodo del árbol tiene un método `visit_NodoX()` correspondiente que evalúa la expresión y retorna un valor inmutable. El entorno de ejecución (*Memdoz*) almacena los *bindings* de variables sin permitir reasignación, garantizando la pureza funcional.

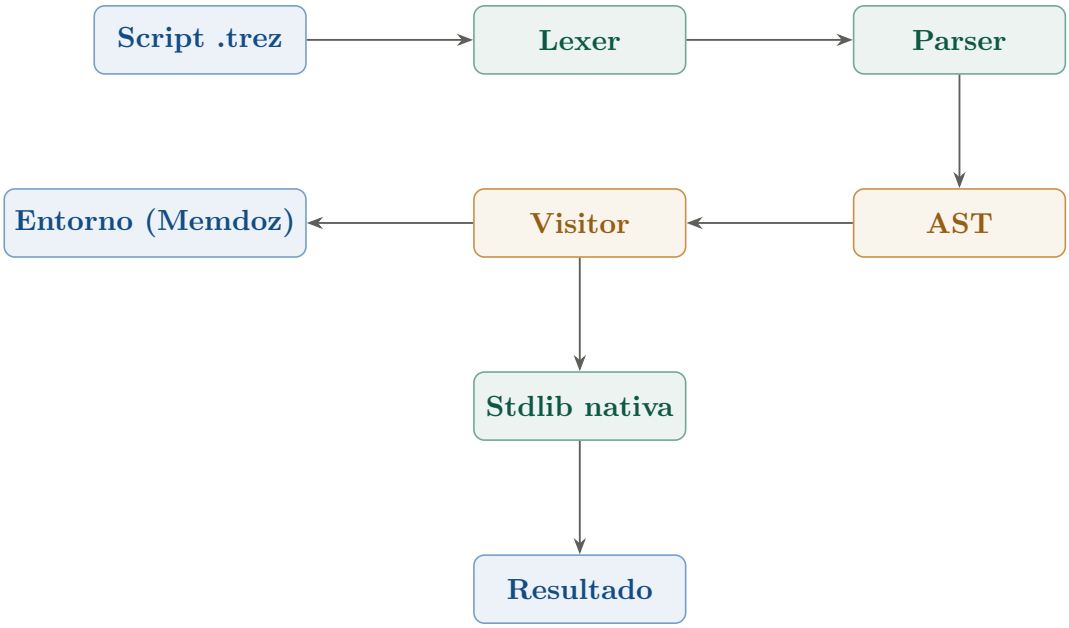

Figura 1: Flujo de ejecución del compilador e intérprete TREZ.

3. Alcance del Proyecto (MVP)

El desarrollo es realizado por un equipo estudiantil de la Universidad Sergio Arboleda. Se define un Producto Mínimo Viable (MVP) que demuestre la viabilidad de un DSL funcional para Deep Learning sin incurrir en optimizaciones de hardware ni en ingeniería de sistemas de producción.

3.1. Dentro del Alcance

Dentro del Alcance (In-Scope)

- **Gramática (ANTLR4) — *Entrega 1 completada, Entrega 2 pendiente*:** `TrezLexer.g4` y `TrezParser.g4` con aritmética, arrays recursivos 1D/2D/3D, `let/;` y llamadas a funciones globales. La Entrega 2 agrega `|>`, namespace `Modulo.func()`, lambdas y desestructuración de tuplas.
- **Motor de evaluación (Visitor) — *Entrega 1 completada*:** `TrezVisitor` con entorno de ejecución, aritmética vectorial, jerarquía

de errores `TrezError` $\supset$ {`TrezSyntaxError`, `TrezRuntimeError`} y `TrezErrorListener` integrado. Las excepciones semánticas diferenciadas (`ShapeMismatchError`, `MathDomainError`) son objetivo de la Entrega 3.

- **Librería estándar nativa — *parcialmente implementada*:** `Mathdoz` (completo), `Tensordoz` (`dot` + `transpose`), `Metricsdoz` (`mse` + `mse_grad`), `IOdoz` (texto plano). Pendientes: `Optimdoz`, `Plotdoz`, `Inspectdoz` y funciones restantes de `Tensordoz` y `Metricsdoz`.
- **Arquitectura MLP entrenable — *Entrega 4*:** Red densa multicapa expresable en sintaxis TREZ con `|>` y módulo `DLdoz`. La base matemática (dot, activaciones, mse, aritmética vectorial) ya está disponible.
- **Autoencoder básico — *Entrega Final*:** Encoder + Decoder encadenados con `|>`. Requiere Entregas 2 y 3 completas.
- **Suite de pruebas — *parcialmente implementada*:** Tests funcionales en `tests/features/` cubren aritmética, tensores, activaciones, backprop manual, IO y errores. Los tests de entrenamiento completo son objetivo de la Entrega 4.

3.2. Fuera del Alcance

Fuera del Alcance (Out-of-Scope)

- **Aceleración GPU/TPU:** Toda ejecución ocurre en CPU.
- **AutoGrad dinámico:** No se construye un grafo computacional dinámico. Las derivadas se proveen analíticamente para las arquitecturas predefinidas.
- **Compilación JIT:** El lenguaje es estrictamente interpretado.
- **Redes Convolucionales (CNN) y Recurrentes (RNN):** Arquitecturas avanzadas quedan como trabajo futuro.
- **Módulos NLPdoz y Visiondoz:** Especificados en el documento de diseño, pero excluidos del MVP.
- **Batching distribuido:** Sin paralelismo de datos ni data loaders asíncronos.

3.3. Hitos de Desarrollo

Entrega	Objetivo
Entrega 1 (completada)	Aritmética básica (`+`, `-`, `*`, `/`) con precedencia y paréntesis; **aritmética vectorial** (escalar×lista, suma y resta elemento a elemento); arrays literales 1D, **2D y 3D** (recursivos); `let`/`;`; constantes `PI`/`E`; funciones nativas: `relu`, `sigmoid`, `dot`, `transpose`, `mse`, `mse_grad`, `exp`, `log`, `sin`, `cos`, `tan`, `sqrt`, `pow`, `factorial`; IO texto (`leer`/`escribir`); jerarquía de errores `TrezError` ⊃ `TrezSyntaxError` + `TrezRuntimeError`; `TrezErrorListener`; `autograd.py` (adelantado).
Entrega 2	Gramática completa: operador `|>` (PIPE token + `pipeOp`), llamadas con namespace `Modulo.func()` (`moduleCall`), lambdas `\x -> e` (`lambdaDef`), desestructuración de tuplas `let [W, b] = expr` (`bindTuple`) y módulo `Inspectdoz`.
Entrega 3	Completar `Tensordoz` (`reshape`, `flatten`, `add`, `concat`), `Mathdoz.softmax`, módulo `Optimdoz` (SGD, Adam), `Metricsdoz` completo e `IOdoz.read_csv` que parsee a tensor.
Entrega 4	MLP entrenable en TREZ + suite de pruebas con dataset CSV real.
Entrega Final	Autoencoder + `Plotdoz.learning_curve` + documentación completa.

Cuadro 1: Cronograma de entregas del proyecto TREZ.

4. Gramática Tipo 2 (ANTLR4)

La gramática de TREZ es **Libre de Contexto** (Tipo 2 en la jerarquía de Chomsky). Se define en el archivo `TREZ.g4` y comprende reglas sintácticas para expresar todo el vocabulario del lenguaje, desde asignaciones simples hasta pipelines de Deep Learning de múltiples pasos.

4.1. Descripción de las Reglas Productivas

`prog`

Punto de entrada del programa. Un programa es una secuencia de una o más

sentencias (`stat+`).

`stat`

Una sentencia puede ser: una asignación simple (`ID = expr`), una asignación de tupla (`[W, b] = expr`), o una expresión evaluada por su efecto (`Plotdoz.learning_curve(...)`)

`expr`

El núcleo del lenguaje. Una expresión puede ser:

- Una **lambda**: `\x -> expr`, que define una función anónima de primer orden.
- Un **pipe**: `expr |> expr`, que aplica el resultado de la izquierda como argumento de la derecha.
- Una **llamada a módulo**: `Tensordoz.dot(A, B)`, con *namespace* explícito.
- Una **llamada a función**: `relu(x)`, sin *namespace*.
- Un **literal de tensor**: `[[1.0, 0.0], [0.0, 1.0]]`.
- Operaciones aritméticas con precedencia correcta (`*`, `/`, `^` antes que `+`, `-`).

`args`

Lista de argumentos separados por comas para llamadas a funciones y módulos.

4.2. Gramática Actual — Entrega 1 (implementada)

La primera entrega implementa aritmética, variables, arrays y llamadas a funciones globales. El separador de sentencias es el punto y coma `;`.

```
// --- TrezLexer.g4 ------------------------------------------------------------
lexer grammar TrezLexer;

LET    : 'let';
LPAREN : '(';  RPAREN : ')';
LBRACK : '[';  RBRACK : ']';
COMMA  : ',';  SEMI   : ';';  EQ : '=';
PLUS   : '+';  MINUS  : '-';  MUL : '*';  DIV : '/';

NUMBER  : '-'? [0-9]+ ('.' [0-9]+)?;
STRING  : '"' (~["\r\n\\] | '\\' .)* '"';
ID      : [a-zA-Z_][a-zA-Z0-9_]*;
WS      : [ \t\r\n]+ -> skip;
COMMENT : '//' ~[\r\n]* -> skip;
```

```

// --- TrezParser.g4 ----------------------------------------------------------

parser grammar TrezParser;
options { tokenVocab = TrezLexer; }

program  : statement+ EOF;

statement : let_stmt
           | expr_stmt
           ;

let_stmt  : LET ID EQ expr SEMI;
expr_stmt : expr SEMI;

expr : expr (MUL | DIV) expr          # MulDivExpr
     | expr (PLUS | MINUS) expr       # AddSubExpr
     | NUMBER                         # NumExpr
     | STRING                         # StringExpr
     | ID                             # VarExpr
     | LPAREN expr RPAREN             # ParenExpr
     | array                          # ArrayExpr
     | ID LPAREN (expr (COMMA expr)*)? RPAREN  # FuncCallExpr
     ;

array : LBRACK RBRACK
      | LBRACK expr (COMMA expr)* RBRACK
      ;
```

Listing 1: TrezLexer.g4 y TrezParser.g4 – estado actual

4.3. Gramática Objetivo — Entrega 2 (planificada)

La siguiente iteración de la gramática incorpora el operador `|>`, las lambdas, las llamadas con *namespace*, la desestructuración de tuplas y los tensores matriciales. El separador cambia a salto de línea (`NEWLINE`) para una sintaxis más limpia.

```
grammar TREZ;

// --- Regla inicial
```

```
    ------------------------------------------------------------

prog    : stat+ ;

// --- Sentencias
    ------------------------------------------------------------

stat    : LET ID EQ expr NEWLINE                    #
    bindVariable
        | LET '[' ID (',' ID)* ']' EQ expr NEWLINE  #
    bindTuple
        | expr NEWLINE                              #
    evalExpr
        | NEWLINE                                   # blank
        ;

// --- Expresiones (menor a mayor precedencia)
    --------------------------------
expr    : '\\' ID '->' expr                         #
    lambdaDef
        | expr '|>' expr                            #
    pipeOp
        | expr op=('+' | '-') expr                  #
    addSub
        | expr op=('*' | '/' | '%') expr            #
    mulDiv
        | expr '^' expr                             # power
        | ID '.' ID '(' args? ')'                   #
    moduleCall
        | ID '(' args? ')'                          #
    funcCall
        | '[' (row (',' row)*)? ']'                 #
    tensorLiteral
        | '(' expr ')'                              #
    parens
        | ID                                        #
    identifier
        | NUMBER                                    #
    number
        | STRING                                    #
    stringLit
```

```
        ;

// --- Fila de tensor (para literales matriciales) ----------------------------
row     : '[' expr (',' expr)* ']' ;

// --- Lista de argumentos ---------------------------------------------------
args    : expr (',' expr)* ;

// --- Tokens ----------------------------------------------------------------

ID      : [a-zA-Z_][a-zA-Z0-9_]* ;
NUMBER  : [0-9]+ ('.' [0-9]+)? ;
STRING  : '"' .*? '"' ;
NEWLINE : '\r'? '\n' ;
COMMENT : '//' ~[\r\n]* -> skip ;
WS      : [ \t]+ -> skip ;
```

Listing 2: TREZ.g4 – gramatica objetivo Entrega 2

4.4. Operador Pipe (|>)

El operador `|>` es el mecanismo central del lenguaje. Semánticamente, `A |> f` equivale a `f(A)`: el valor del lado izquierdo se pasa automáticamente como primer argumento de la función del lado derecho. Esto permite expresar transformaciones secuenciales sin anidamiento de paréntesis.

> **Precedencia del pipe:** El operador `|>` tiene la **menor precedencia** de todos los operadores. Esto garantiza que `a + b |> f` se evalúa como `f(a + b)`, no como `a + f(b)`.

5. Descripción Sintáctica del Lenguaje

Esta sección describe la sintaxis concreta de TREZ a través de sus construcciones fundamentales, junto con ejemplos escritos en el propio lenguaje.

5.1. Asignación de Variables

La asignación en TREZ es inmutable: una vez ligado un nombre a un valor, no puede reasignarse dentro del mismo entorno de ejecución. En la Entrega 1, la sintaxis usa la palabra clave `let` y el punto y coma `;` como terminador. A partir de la Entrega 2, se soportará además la desestructuración de tuplas.

```
// Sintaxis actual (Entrega 1)
let tasa_aprendizaje = 0.01;
let epocas = 100;
let pesos = [0.5, 0.3, 0.1];
let matriz = [[1, 2], [3, 4]];

// Resultado de expresion (se imprime automaticamente)
pesos;
```

Listing 3: Asignacion – sintaxis Entrega 1 (implementada)

```
// Desestructuracion de tupla (Entrega 2)
let [X_train, X_test] = ModelSeldoz.split(X, Y, 0.8);
let [W1, b1, W2, b2]  = DLdoz.init_dense(4, 16);
```

Listing 4: Asignacion de tupla – sintaxis Entrega 2 (planificada)

5.2. Funciones Lambda

Las lambdas en TREZ son funciones anónimas de primer orden, planificadas para la **Entrega 2**. Se definen con la sintaxis `\param -> cuerpo` y pueden pasarse como argumentos a otras funciones o al operador `|>`.

```
// Lambda que aplica ReLU elemento a elemento
let activar = \x -> Mathdoz.relu(x);

// Lambda para actualizar pesos con SGD
let actualizar = \W -> Optimdoz.sgd(W, grad_W, 0.01);

// Pasar lambda como argumento a Pipeline.map
let datos_norm = Pipelinedoz.map(dataset, \x -> Datadoz.
   normalize(x));
```

Listing 5: Definicion y uso de lambdas (Entrega 2)

5.3. Llamadas a Módulos (*Namespace*)

Todas las funciones de la librería estándar se invocan mediante la notación de módulo `Modulo.funcion(args)`. Esta convención hace explícito el origen de cada operación y evita colisiones de nombres.

```
// Algebra lineal (Entrega 2: namespace Tensordoz.func)
let producto    = Tensordoz.dot(W, X);
let transpuesta = Tensordoz.transpose(W);

// Activaciones
let salida  = Mathdoz.sigmoid(Tensordoz.dot(W, X));
let logits  = Mathdoz.softmax(Tensordoz.dot(W2, h1));

// Metricas
let error    = Metricsdoz.mse(Y_real, Y_pred);
let exactitud = Metricsdoz.accuracy(Y_real, Y_pred);

// Equivalente en Entrega 1 (funciones globales, ya
   implementado)
let prod_e1   = dot(W, X);
let error_e1  = mse(Y_real, Y_pred);
```

Listing 6: Llamadas a modulos – Entrega 2 (namespace planificado)

5.4. Literales de Tensor

Los tensores se expresan como listas anidadas de literales numéricos. El intérprete infiere la forma (*shape*) del tensor a partir de la estructura de anidamiento.

Disponible desde la Entrega 1: aunque el documento anterior planeaba los tensores 2D y 3D para la Entrega 2, la regla `array` del parser es recursiva (`array` puede contener `expr`, y `expr` puede ser otro `array`). Por esto, matrices y tensores 3D **ya se parsean y evalúan correctamente** en la Entrega 1 actual.

```
// Vector (shape: [3])
let bias = [0.1, 0.2, 0.3];

// Matriz (shape: [2, 3]) -- ya funciona en Entrega 1
let pesos = [[0.5, 0.1, 0.3],
             [0.2, 0.8, 0.4]];
```

```

// Tensor 3D (shape: [2, 2, 2]) -- ya funciona en Entrega 1
let kernel = [[[1.0, 0.0], [0.0, 1.0]],
              [[0.5, 0.5], [0.5, 0.5]]];

// Uso en operacion tensorial
let x = [[1, 2], [3, 4]];
let y = [[2, 0], [1, 2]];
dot(x, y);
```

Listing 7: Literales de tensor – funciona en Entrega 1

5.5. Pipelines con el Operador |>

El operador de composición |> encadena transformaciones funcionales. Cada etapa del pipeline recibe el resultado de la etapa anterior como su primer argumento.

```
// Pipeline de preprocesamiento
X_listo = IOdoz.read_csv("datos.csv")
    |> Datadoz.drop_na()
    |> Datadoz.drop_duplicates()
    |> Datadoz.normalize()

// Pipeline de inferencia con inspeccion
prediccion = X_test
    |> Inspectdoz.spy()
    |> \x -> Tensordoz.dot(W1, x)
    |> Mathdoz.relu()
    |> \h -> Tensordoz.dot(W2, h)
    |> Mathdoz.softmax()
```

Listing 8: Pipeline de preprocesamiento y entrenamiento

6. Librería Estándar Nativa para Deep Learning

La totalidad de las operaciones matemáticas y tensoriales están implementadas en Python puro, sin dependencias externas. A continuación se describe cada módulo con sus funciones mínimas requeridas para el MVP.

6.1. Módulo `Tensordoz` – Álgebra Lineal

De las funciones listadas, `dot` y `transpose` están implementadas en la Entrega 1 dentro de `tensor_mathdoz.py`. Las demás (`reshape`, `flatten`, `add`, `concat`) son objetivo de la Entrega 3.

Función	**Descripción**
`Tensordoz.dot(A, B)`	Producto matricial. Lanza `ShapeMismatchError` si las dimensiones son incompatibles.
`Tensordoz.transpose(A)`	Transpone un tensor 2D.
`Tensordoz.reshape(A, shape)`	Reorganiza los elementos sin copiar datos.
`Tensordoz.flatten(A)`	Convierte el tensor a un vector 1D.
`Tensordoz.add(A, B)`	Suma elemento a elemento con soporte de *broadcasting* básico.
`Tensordoz.mul_scalar(A, k)`	Multiplica todos los elementos por un escalar.
`Tensordoz.concat([A, B], axis)`	Concatena tensores a lo largo de un eje.

Cuadro 2: Funciones del módulo `Tensordoz`.

6.2. Módulo `Mathdoz` – Matemáticas y Activaciones

Implementadas en la Entrega 1: `relu`, `sigmoid`, `exp`, `log`, `sin`, `cos`, `tan`, `sqrt`, `pow`, `factorial`, constantes `PI` y `E` (en `core_mathdoz.py` y `activationsdoz.py`). Pendientes para la Entrega 3: `softmax` y `derivative`.

Función	**Descripción**
`Mathdoz.relu(x)`	$\text{máx}(0, x)$ aplicado elemento a elemento.
`Mathdoz.sigmoid(x)`	$\sigma(x) = \frac{1}{1+e^{-x}}$ aplicado elemento a elemento.
`Mathdoz.softmax(x)`	Distribución de probabilidades sobre un vector.
`Mathdoz.exp(x)`	Exponencial elemento a elemento.
`Mathdoz.log(x)`	Logaritmo natural. Lanza `MathDomainError` si $x \leq 0$.
`Mathdoz.derivative(f, x)`	Derivada analítica para funciones de activación predefinidas.

Cuadro 3: Funciones del módulo `Mathdoz`.

6.3. Módulo `Optimdoz` – Optimizadores

Los optimizadores actualizan los parámetros de la red en cada paso de entrenamiento. En TREZ, la actualización se expresa como una función pura que recibe los pesos actuales y retorna los pesos actualizados. Este módulo es objetivo de la **Entrega 3** — no existe aún en el repositorio como módulo independiente. La actualización de pesos se demuestra en los tests actuales mediante aritmética directa: `let w_new = w - 0.01 * grad;`.

Función	**Descripción**
`Optimdoz.sgd(W, grad, lr)`	Descenso de gradiente estocástico: $W \leftarrow W - \eta \cdot \nabla W$.
`Optimdoz.adam(W, grad, m, v, t, lr)`	Optimizador Adam con estimadores de primer y segundo momento.

Cuadro 4: Funciones del módulo `Optimdoz`.

6.4. Módulo `Metricsdoz` – Evaluación

Implementadas en la Entrega 1: `mse` y `mse_grad` (en `lossesdoz/lossdoz.py`). Las demás métricas (`cross_entropy`, `accuracy`, `rmse`, `r2_score`) son objetivo de la Entrega 3.

Función	**Descripción**
`Metricsdoz.mse(Y, Yp)`	Error Cuadrático Medio: $\frac{1}{n}\sum(y_i - \hat{y}_i)^2$.
`Metricsdoz.cross_entropy(Y, Yp)`	Pérdida de entropía cruzada para clasificación.
`Metricsdoz.accuracy(Y, Yp)`	Proporción de predicciones correctas.
`Metricsdoz.rmse(Y, Yp)`	Raíz del Error Cuadrático Medio.
`Metricsdoz.r2_score(Y, Yp)`	Coeficiente de determinación R^2.

Cuadro 5: Funciones del módulo `Metricsdoz`.

6.5. Módulo `IOdoz` – Entrada/Salida

En la Entrega 1, `IOdoz` expone dos funciones de texto plano: `leer(ruta)` lee el contenido completo de un archivo y lo retorna como cadena, y `escribir(ruta, contenido)` escribe o sobreescribe un archivo. La función `IOdoz.read_csv(ruta)`, que parsea un CSV directamente a una lista de listas numéricas (tensor), es objetivo de la Entrega 3.

Función	Descripción
`leer(ruta)`	Lee el contenido de un archivo de texto y lo retorna como cadena. **Implementado.**
`escribir(ruta, contenido)`	Escribe o sobreescribe un archivo con el contenido dado. **Implementado.**
`IOdoz.read_csv(ruta)`	Parsea un CSV a tensor (lista de listas numéricas). *Planificado Entrega 3.*

Cuadro 6: Funciones del módulo `IOdoz`.

6.6. Módulo `Plotdoz` – Visualización

Este módulo es objetivo de la **Entrega Final** y aún no existe en el repositorio. Todas las funciones listadas son planificadas.

Función	Descripción
`Plotdoz.learning_curve(epochs, loss_t, loss_v)`	Curva de convergencia para monitorear el decaimiento del error y detectar sobreajuste.
`Plotdoz.histogram(X, bins)`	Distribución de frecuencias de una variable.
`Plotdoz.boxplot(X)`	Diagrama de caja para detección de valores atípicos.
`Plotdoz.correlation_heatmap(X)`	Mapa de calor de la matriz de correlación de Pearson.

Cuadro 7: Funciones del módulo `Plotdoz`.

6.7. Módulo `Inspectdoz` – Depuración No Invasiva

Módulo planificado para la **Entrega 2**, junto con la implementación del operador `|>`. Su función `spy` es especialmente útil en pipelines porque retorna el tensor intacto, permitiendo observar el flujo sin romperlo.

Función	Descripción
`Inspectdoz.shape(tensor)`	Imprime en consola las dimensiones del tensor (ej. `[128, 64]`).
`Inspectdoz.spy(tensor)`	Imprime el tensor **y lo retorna sin modificarlo**. Ideal para insertar en medio de un `|>` sin romper el pipeline.

Cuadro 8: Funciones del módulo `Inspectdoz`.

7. Sistema de Manejo de Errores

En el desarrollo de modelos de Deep Learning, la gran mayoría de los fallos en tiempo de ejecución provienen de incompatibilidades dimensionales. TREZ implementa un sistema de excepciones semánticas que detecta estos errores con mensajes descriptivos.

7.1. Excepciones Definidas

El sistema de errores de TREZ diferencia dos categorías: errores de sintaxis, detectados durante el análisis por ANTLR4, y errores en tiempo de ejecución, detectados durante la evaluación del AST por el Visitor. Ambas categorías están implementadas desde la Entrega 1. Las excepciones semánticas *diferenciadas* (por tipo de operación) son objetivo de las Entregas 2 y 3.

`TrezError` (*implementado*)
: Clase base de la jerarquía de errores en `errors.py`. Todas las excepciones de TREZ heredan de ella.

`TrezSyntaxError` (*implementado*)
: Se lanza durante el análisis léxico/sintáctico. Incluye número de línea, número de columna y descripción del token inesperado. Es capturada por `TrezErrorListener`, que se adjunta al *Lexer* y al *Parser* en `main.py`. Si se detectan errores sintácticos, la ejecución se detiene antes de visitar el AST.

```
// tests/syntax/test_syntax_error.trez
let arr = [1, 2,,];    // Error: coma doble
x = 5                  // Error: falta "let" y ";"
```

Listing 9: Ejemplo – TrezSyntaxError en test de sintaxis

`TrezRuntimeError` (*implementado*)

Excepción semántica base. Cubre variables indefinidas (*"Undefined variable"*), dimensiones incompatibles en `dot` (*"Dimension mismatch"*), logaritmo de valor negativo y errores de IO. El mensaje describe la causa en texto plano.

```
// tests/runtime/test_runtime_error.trez
let a = [1, 2, 3];
let b = [4, 5];          // Longitud distinta
let err = dot(a, b);     // -> TrezRuntimeError: Dimension
    mismatch
```

Listing 10: Ejemplo – TrezRuntimeError en test de runtime

`ShapeMismatchError` (*planificado Entrega 3*)

Reemplazará al mensaje genérico de `TrezRuntimeError` para incompatibilidades dimensionales. Incluirá las formas concretas de ambos operandos.

`MathDomainError` (*planificado Entrega 3*)

Reemplazará al mensaje genérico para operaciones ilegales como `log(0)`.

`UndefinedSymbolError` (*planificado Entrega 2*)

Reemplazará al mensaje genérico *"Undefined variable"* con información del símbolo y el entorno actual.

8. Ejemplos de Programas Completos en TREZ

8.1. Caso 0: Backpropagation Manual (Entrega 1 – ejecutable hoy)

Este script está tomado directamente del archivo `tests/features/test_backprop.trez` del repositorio y funciona con la Entrega 1. Demuestra que la aritmética vectorial del Visitor permite implementar el paso de actualización de pesos sin necesidad del módulo `Optimdoz`.

```
// Funcion de perdida MSE
let predicciones = [0.1, 0.9, 0.4];
let reales       = [0.0, 1.0, 0.5];

let error      = mse(reales, predicciones);
let gradientes = mse_grad(reales, predicciones);

error;
gradientes;
```

```

// Actualizacion de pesos: W_nuevo = W_viejo - lr * gradiente
// Funciona gracias a la aritmetica vectorial del Visitor
let pesos       = [0.5, 0.5, 0.5];
let pesos_nuevos = pesos - 0.1 * gradientes;

pesos_nuevos;
```

Listing 11: test_backprop.trez – funciona en Entrega 1

8.2. Caso 1: Entrenamiento de una Red Neuronal MLP

Este ejemplo muestra el ciclo de entrenamiento completo que TREZ podrá ejecutar al completar la Entrega 4. Utiliza sintaxis de las Entregas 2 y 3: operador `|>`, namespace `Modulo.func()` y módulo `DLdoz`.

```
// -- 1. Carga y preprocesamiento ----------------------------------------
dataset = IOdoz.read_csv("datos_iris.csv")
    |> Datadoz.drop_na()
    |> Datadoz.normalize()

[X_train, X_test] = ModelSeldoz.split(dataset, 0.8)
Y_train = Tensordoz.select(X_train, ["clase"])
X_train = Tensordoz.select(X_train, ["sepal_l", "sepal_w", "petal_l", "petal_w"])

// -- 2. Inicializacion de parametros -------------------------------------
[W1, b1] = DLdoz.init_dense(4, 16)
[W2, b2] = DLdoz.init_dense(16, 3)

// -- 3. Forward pass ------------------------------------------------------
forward = \X ->
    X
    |> \x -> Tensordoz.dot(W1, Tensordoz.transpose(x))
    |> \z -> Tensordoz.add(z, b1)
    |> Mathdoz.relu()
    |> \h -> Tensordoz.dot(W2, h)
    |> \z -> Tensordoz.add(z, b2)
```

```
    |> Mathdoz.softmax()

// -- 4. Ciclo de entrenamiento (100 epocas) ---------------------------------
[W1, W2, hist_train, hist_val] = DLdoz.train_loop(
    forward, X_train, Y_train, X_test, Y_test,
    Metricsdoz.cross_entropy, Optimdoz.adam, 0.001, 100
)

// -- 5. Evaluacion final ----------------------------------------------------
Y_pred     = forward(X_test)
exactitud  = Metricsdoz.accuracy(Y_test, Y_pred)

// -- 6. Visualizacion -------------------------------------------------------
Plotdoz.learning_curve(hist_train, hist_val)
```

Listing 12: entrenamiento_mlp.trez – MLP para clasificacion binaria

8.3. Caso 2: Autoencoder para Reducción de Dimensionalidad

Ejemplo objetivo de la Entrega Final. Demuestra cómo las lambdas y el operador |> permiten expresar encoder y decoder como funciones puras componibles.

```
// -- 1. Datos ----------------------------------------------------------------

X = IOdoz.read_csv("mnist_flat.csv")
    |> Datadoz.normalize()

// -- 2. Parametros ----------------------------------------------------------

[W_enc, b_enc] = DLdoz.init_dense(784, 64)
[W_dec, b_dec] = DLdoz.init_dense(64, 784)

// -- 3. Encoder y Decoder como funciones puras ------------------------------
encoder = \x ->
    x
```

```
    |> \i -> Tensordoz.dot(W_enc, i)
    |> \z -> Tensordoz.add(z, b_enc)
    |> Mathdoz.relu()

decoder = \h ->
    h
    |> \i -> Tensordoz.dot(W_dec, i)
    |> \z -> Tensordoz.add(z, b_dec)
    |> Mathdoz.sigmoid()

// -- 4. Pipeline completo: comprimir y reconstruir ------------------------
autoencoder = \x -> x |> encoder |> decoder

// -- 5. Entrenamiento con MSE como perdida de reconstruccion ---------------
[W_enc, b_enc, W_dec, b_dec, hist] = DLdoz.train_loop(
    autoencoder, X, X,
    Metricsdoz.mse, Optimdoz.adam, 0.001, 50
)

// -- 6. Inspeccion del espacio latente -------------------------------------
codigo_latente = X |> encoder |> Inspectdoz.shape()

// -- 7. Curva de reconstruccion ---------------------------------------------
Plotdoz.learning_curve(hist, hist)
```

Listing 13: autoencoder.trez – Autoencoder denso 784-64-784

8.4. Caso 3: Análisis Exploratorio con Depuración de Pipeline

Ejemplo que demuestra `Inspectdoz.spy()` en acción dentro de un pipeline, planificado para la Entrega 2. En la Entrega 1, la inspección se realiza evaluando expresiones directamente: `pesos;` imprime el valor en consola.

```
// Inspectdoz.spy() permite "asomarse" al pipeline sin romperlo
datos_limpios = IOdoz.read_csv("financiero.csv")
    |> Datadoz.drop_duplicates()
```

```
    |> Inspectdoz.spy()          // imprime shape en consola
    |> Datadoz.drop_na()
    |> Datadoz.normalize()
    |> Inspectdoz.spy()          // verifica shape final

X = Tensordoz.select(datos_limpios, ["ingreso", "deuda", "edad"])

// Visualizaciones analiticas
Plotdoz.boxplot(Tensordoz.slice(X, 0, 1, 1))
Plotdoz.correlation_heatmap(X)
```

Listing 14: exploratorio.trez – EDA con inspeccion no invasiva

9. Diagramas del AST y el Patrón Visitor

Esta sección ilustra con precisión cómo TREZ transforma un programa fuente en resultados concretos, pasando por tres etapas: la construcción del *Parse Tree* por ANTLR4, la generación del AST simplificado, y el recorrido del AST mediante el patrón *Visitor*.

9.1. Etapa 1 – Del Código Fuente al Parse Tree

Dado el fragmento de código TREZ:

```
resultado = X |> Mathdoz.relu()
```

Listing 15: Expresion fuente de ejemplo para los diagramas

ANTLR4 produce el **Parse Tree** completo aplicando la gramática `TREZ.g4`. Cada nodo interno corresponde a una regla gramatical; las hojas son los tokens terminales reconocidos por el *Lexer*.

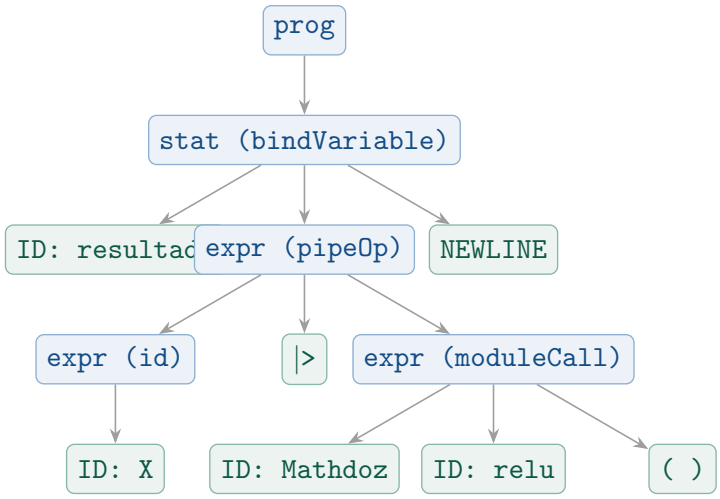

Figura 2: Parse Tree generado por ANTLR4 para `resultado = X |> Mathdoz.relu()`.

9.2. Etapa 2 – Del Parse Tree al AST

El **Árbol Sintáctico Abstracto** (AST) elimina los nodos redundantes del Parse Tree (tokens de puntuación, nodos intermedios triviales) y retiene únicamente la estructura semánticamente relevante. Cada nodo del AST es un objeto Python con un tipo y sus hijos directos.

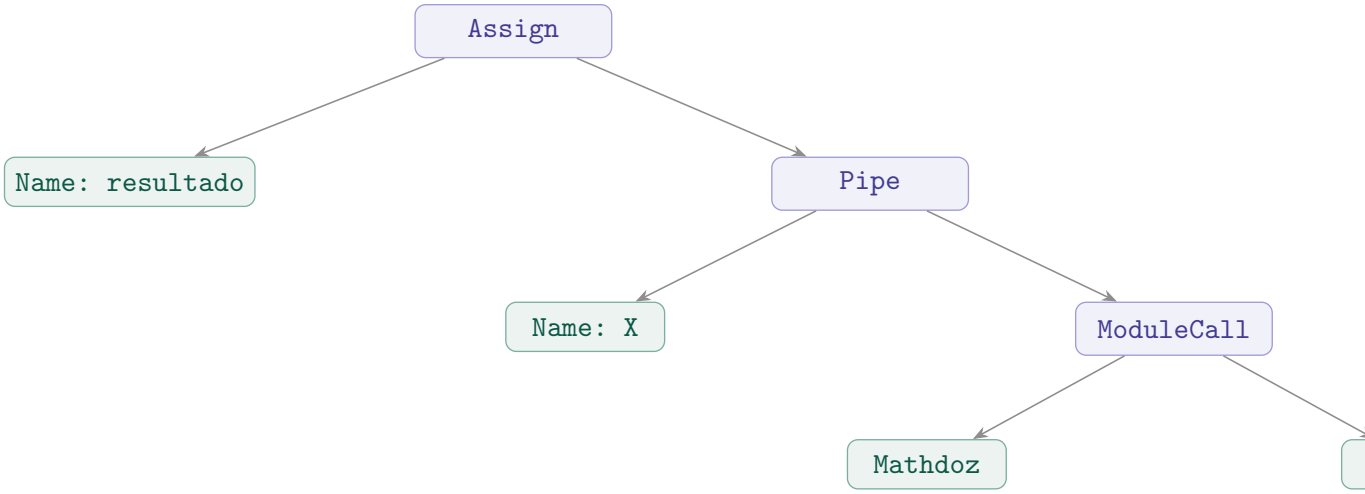

Figura 3: AST simplificado derivado del Parse Tree anterior.

Comparando las Figuras 2 y 3, el AST elimina los tokens `|>`, el punto de acceso al módulo, los paréntesis y el `NEWLINE`, conservando únicamente los nodos con significado semántico: `Assign`, `Pipe`, `Name` y `ModuleCall`.

9.3. Etapa 3 – Recorrido del AST con el Patrón Visitor

El patrón **Visitor** separa la estructura del árbol de la lógica de evaluación. La clase `TREZVisitor` en Python define un método `visit_NodoX()` por cada tipo de nodo del AST. El recorrido es *post-order*: primero se visitan los hijos y luego el nodo padre opera sobre sus resultados ya evaluados.

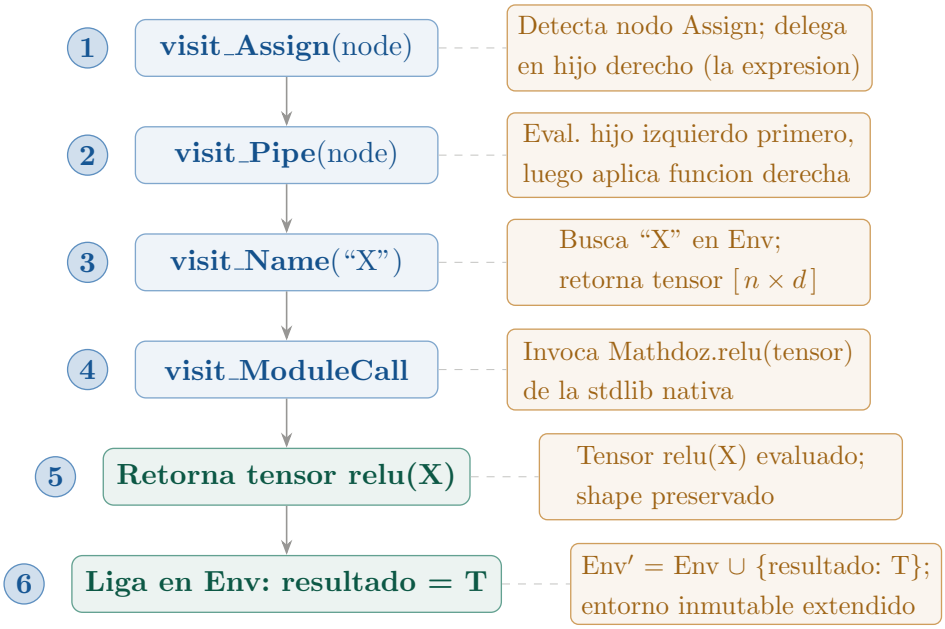

Figura 4: Secuencia de visitas del Visitor al evaluar `resultado = X |> Mathdoz.relu()`.

9.4. Etapa 4 – Flujo Completo de un Pipeline de Entrenamiento

El siguiente diagrama muestra cómo un pipeline multi-etapa es descompuesto en nodos `Pipe` anidados dentro del AST, y cómo el Visitor los recorre de izquierda a derecha propagando el tensor transformado en cada paso.

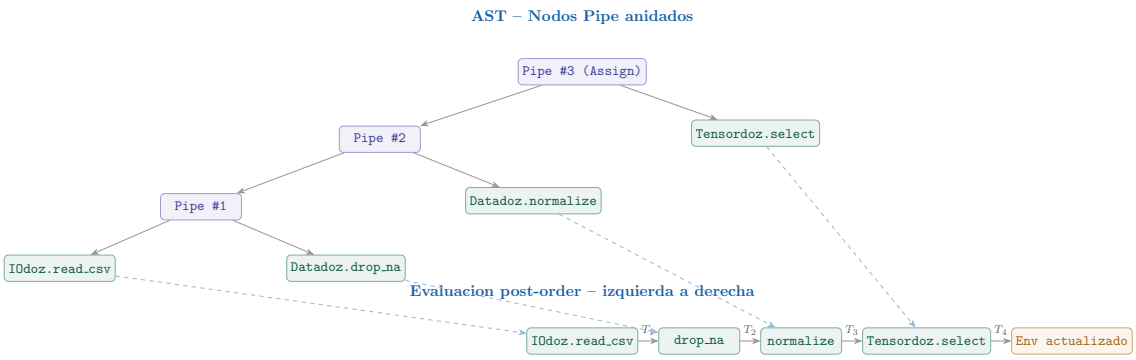

Figura 5: AST de un pipeline de cuatro etapas y su orden de evaluación post-order por el Visitor.

Invariante del Visitor: cada nodo `Pipe` recibe exactamente un tensor de entrada y retorna exactamente un tensor de salida. Esta propiedad garantiza que el `ShapeMismatchError` puede ser detectado y reportado con precisión en cualquier etapa del pipeline, indicando el número de paso y las dimensiones concretas del tensor incompatible.

10. Semántica Operacional del Intérprete

El intérprete de TREZ implementa una semántica de **evaluación estricta** (también llamada *eager evaluation* o *call-by-value*): los argumentos de una función se evalúan completamente antes de aplicar la función. Esto garantiza un orden de evaluación predecible, esencial para el seguimiento del flujo de datos en pipelines de entrenamiento.

10.1. Reglas de Evaluación

Asignación (`bindVariable`)

Se evalúa el lado derecho, se liga el resultado al identificador en el entorno actual y se retorna el entorno extendido. El entorno es inmutable: no se puede modificar una ligadura existente.

Multiplicación y división (`MulDivExpr`)

Además de escalares, soporta **aritmética vectorial**: `escalar * lista` y `lista * escalar` aplican la operación a cada elemento. Esto permite expresar la actualización de pesos directamente: `let w_new = w - 0.1 * grad;`.

Suma y resta (`AddSubExpr`)

Cuando ambos operandos son listas de igual longitud, opera **elemento a elemento**. Si las longitudes difieren, lanza `TrezRuntimeError`. Esto hace posible el patrón de backpropagation sin necesidad del módulo `Optimdoz`.

Pipe (`pipeOp`)

Se evalúa primero la expresión izquierda, obteniendo un valor v. Luego se evalúa la expresión derecha, que debe resultar en una función f. Finalmente se aplica $f(v)$.

Lambda (`lambdaDef`)

La expresión `\x -> e` produce un *closure* que captura el entorno léxico en el momento de la definición. No se evalúa el cuerpo hasta la aplicación.

Llamada a módulo (`moduleCall`)

Se busca la función `funcion` dentro del módulo `Modulo` en la librería estándar nativa. Se evalúan todos los argumentos y se invoca la implementación Python correspondiente.

11. Conclusiones y Trabajo Futuro

TREZ demuestra que es posible construir un lenguaje funcional especializado en Deep Learning sin depender de ninguna librería de terceros. La combinación de ANTLR4 para el análisis sintáctico y el patrón Visitor en Python ofrece una arquitectura clara, extensible y educativamente valiosa.

El MVP planteado cubre el ciclo completo de un proyecto de Machine Learning: ingesta de datos, preprocesamiento, definición de arquitectura, entrenamiento, evaluación de métricas y visualización de resultados, todo expresable en la sintaxis declarativa del lenguaje.

Como trabajo futuro, dentro del propio MVP (Entregas 2–Final), se identifican:

- **Operador `|>` y namespace (Entrega 2):** Es la brecha más urgente. Sin el pipe, ninguno de los scripts de ejemplo puede ejecutarse. Requiere actualizar ambos archivos `.g4`, agregar `PIPE`, `DOT` al Lexer y los nodos `visitPipeOp`, `visitModuleCall`, `visitLambdaDef` y `visitBindTuple` al Visitor.
- **Completar `Tensordoz` y `Metricsdoz` (Entrega 3):** Agregar `reshape`, `flatten`, `add`, `concat` al módulo tensorial, y `cross_entropy`, `accuracy`, `rmse`, `r2_score` a métricas.
- **Módulo `Optimdoz` (Entrega 3):** Crear `optimdoz/optimdoz.py` con `sgd` y `adam` como funciones puras.
- **AutoGrad (ya adelantado):** El repositorio ya contiene `autograd.py` con diferenciación automática escalar. Integrarlo al Visitor para usarlo desde sintaxis TREZ es trabajo de la Entrega Final.

Como extensiones post-MVP se identifican:

- **Redes Convolucionales (CNN):** Módulo `Convdoz` con `conv2d`, `max_pool` y `batch_norm`.
- **Módulo NLPdoz:** Tokenización, embeddings y arquitecturas Transformer básicas.
- **Compilación a bytecode:** Etapa de compilación intermedia para mejorar el rendimiento del intérprete.